

Project Report: Comparative Analysis of Classical vs. Neural NLP

Name: Neena S Nair, 24m0153.

Topic: COVID-19 Sentiment Classification (Sentiment Analysis)

Q: Why are tokenization and embeddings necessary, and what happens if tokenization is bad?

1. Why do we need them? (The Logic) Transformers are basically just big mathematical engines. They cannot "read" English or understand text strings like "Hello." They only understand numbers.

- **Tokenization** is the translator. It chops the text into smaller pieces (tokens) and converts them into ID numbers. It's the only way to get raw text into the machine.
- **Embeddings** provide the *meaning*. If we just gave words random ID numbers (e.g., Dog=4, Cat=99), the computer wouldn't know they are related. Embeddings convert those IDs into dense vectors where similar words are mathematically close to each other. This helps the model understand context (like knowing "King" is related to "Queen").

2. What happens if Tokenization is poorly designed? Two main problems arise if the tokenizer is bad:

- **The "Unknown Word" Issue:** If the tokenizer is too rigid, it won't recognize new or rare words. It will just replace them with an <UNK> (Unknown) tag, causing the model to lose important information.
- **Breaking Meaning:** A bad tokenizer might split words illogically (e.g., splitting "unhappiness" into "u-nh-app" instead of "un-happy-ness"). This makes it much harder for the model to learn the root meaning of the word, leading to lower accuracy.

Q2. Build two text classification systems on the same dataset (<https://www.kaggle.com/datasets/datatattle/covid-19-nlp-text-classification>).

1. Objective

To compare the performance of a classical statistical approach (TF-IDF + Logistic Regression) against a modern neural approach (Word Embeddings + Neural Network) on a dataset of COVID-19 tweets. The goal is to determine if the computational complexity of a neural network yields a significant accuracy improvement.

2. Methodology

I developed two distinct systems using Python (Scikit-learn & TensorFlow):

- **System A: Classical NLP (Baseline)**
 - **Technique:** TF-IDF Vectorization (Bag of Words).
 - **Model:** Logistic Regression.
 - **Logic:** This model assigns sentiment based purely on the frequency of specific words, ignoring word order or context.

--- System A: Sentiment Drivers Comparison ---					
	Negative Words	Neg Weight		Positive Words	Pos Weight
0	crisis	12.852604		great	9.795868
1	panic	8.363065		best	9.158420
2	no	7.465327		hand	8.581766
3	scams	6.437319		help	8.213805
4	kill	6.193809		free	8.075560
5	worst	5.764496		like	7.352546
6	stop	5.522217		good	7.177616
7	war	5.501455		love	7.112931
8	hell	5.349200		safe	6.770617
9	sick	5.230634		support	6.572107

Figure A: Comparative Feature Importance.

The table displays the highest weighted keywords for both 'Extremely Negative' (left) and 'Extremely Positive' (right) classes. The contrast between words like *"crisis"* versus *"safe"* demonstrates the model's ability to distinguish opposing sentiment triggers.

- **System B: Neural NLP (Deep Learning)**
 - **Technique:** Learned Word Embeddings (Dense Vector Space).
 - **Model:** Keras Sequential Network (Embedding Layer →Global Average Pooling →Dense Layer).
 - **Logic:** This model maps words to vectors where synonyms are mathematically close, allowing it to capture semantic meaning rather than just keywords.

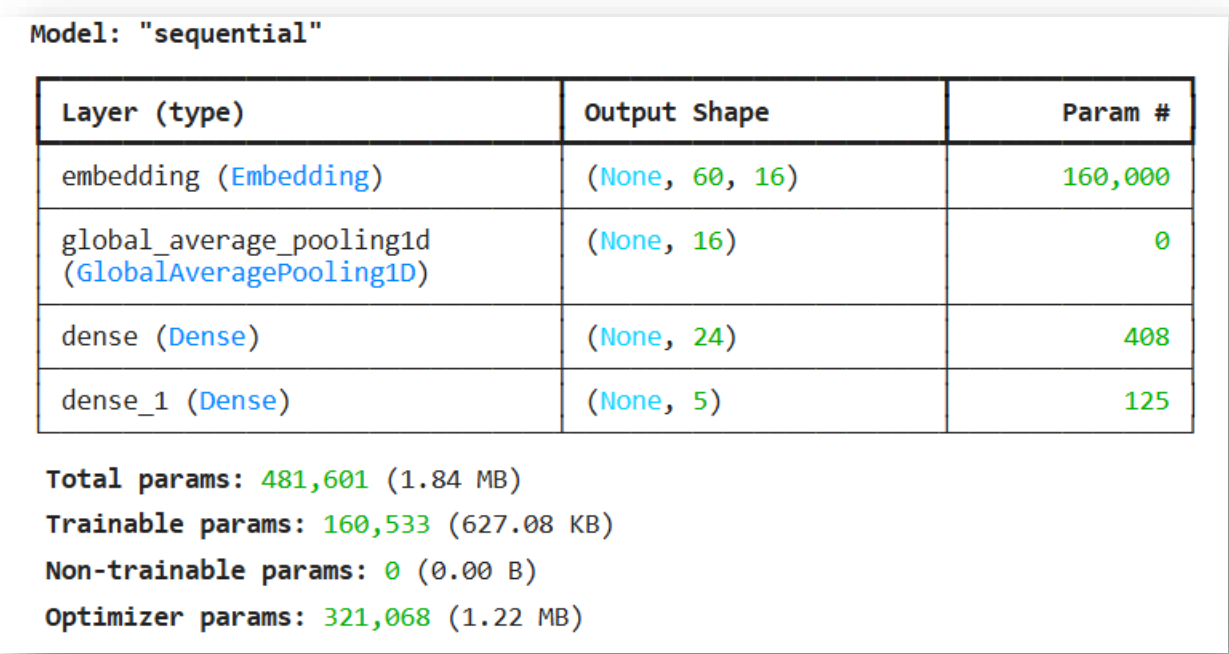


Figure B: System B architecture demonstrating the neural network developed.

3. Experimental Results

System	Method	Accuracy	Training Time
System A	TF-IDF + Logistic Regression	56.40%	< 5 Seconds
System B	Embeddings + Neural Network	66.22%	~30 Seconds

4. Key Observations & Conclusion

The Neural Network (System B) outperformed the Classical baseline by approximately **10%**.

The gap:

The dataset uses fine-grained labels (e.g., *Extremely Positive* vs. *Positive*). System A struggles here because it treats words independently. System B, however, captures **semantic intensity** via embeddings—it "understands" that words like *catastrophe* represent a stronger sentiment than *bad*, allowing it to better distinguish between extreme and moderate classes.

COMPARISON CONFUSION MATRICES BETWEEN TWO SYSTEMS

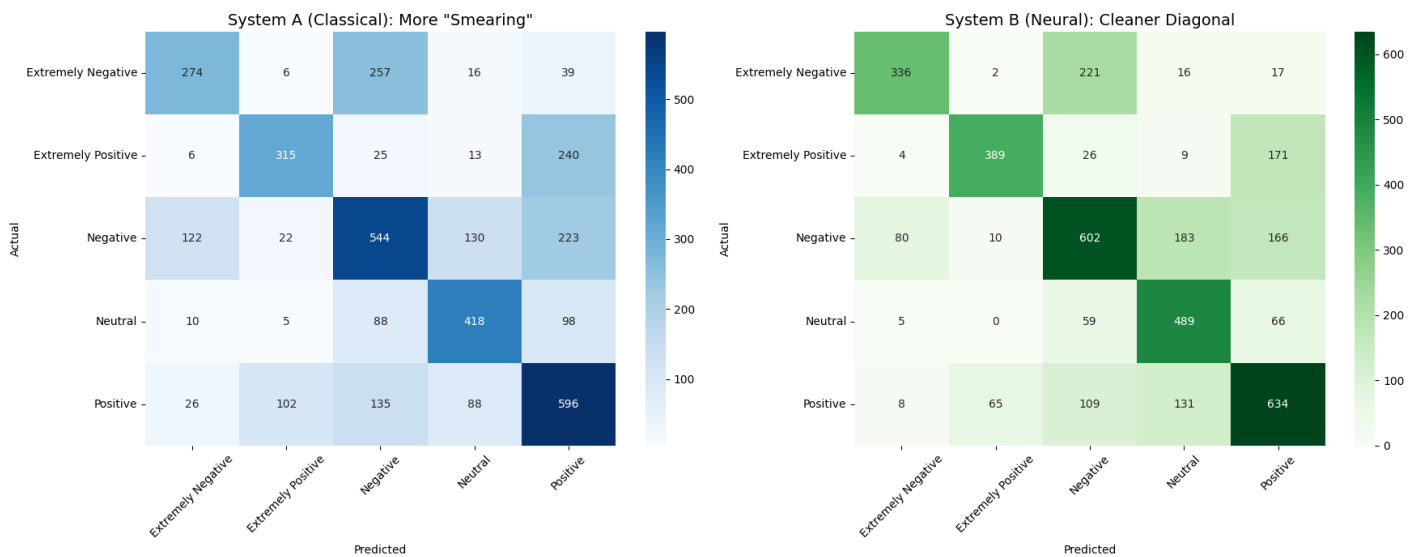


Figure C: Demonstrates the comparison of test results between the two systems A and B.

Conclusion:

While the classical model is faster, the neural approach is superior for this task. The 10% gain in accuracy proves that capturing semantic context is essential for fine-grained sentiment analysis.